

01 / FOUNDATIONS

Capability-Conditioned HetNet

A clear guide to the project and its current evidence

We study how a team of agents with different abilities can cooperate through short messages. An **agent** is a simulated robot. Its **policy** is a learned rule for choosing an action. In **reinforcement learning**, experience and rewards train that rule. The difficulty is that each agent has only part of the information: one may see a target while another can capture it.

The research question: can one shared policy use each agent’s current abilities to coordinate when the team changes or a sensor fails?

The starting point: HetNet

HetNet connects agents in a **communication graph**: agents are nodes, and an edge means one can send a message to another. **Attention** learns how much weight to give each incoming message. The released model uses physical agent classes to select communication transforms; it supports real-valued and binary messages. We retain its simulators and the idea of learning communication together with action selection. The reproduction runs discussed here use **HetNet Real**, with real-valued messages. [\[1\]](#)

The environments, in everyday terms

Task	What the team must do	Team / time limit
PP	Predator–Prey: all three sensing agents reach a stationary prey.	3 sensing / 80 steps
PCP	Predator–Capture: sensing agents reach the prey; the actuation agent reaches and captures it.	2 sensing + 1 acting / 80 steps
FC	FireCommander: locate spreading fire and extinguish all remaining fire cells.	2 sensing + 1 acting / 300 steps

All three recipes use a 5×5 grid. “P” and “A” are the simulator’s names for sensing and actuation classes. Sensing agents can move but lack the special capture/extinguish action; actuation agents can perform it but lack environmental sensing.

What we draw from other work

Capability-aware teamwork: Howell et al. show why a shared policy can benefit from knowing a robot’s abilities. CASH also conditions policy parameters on capabilities. Our use of capabilities is therefore established prior art, not a first-of-its-kind claim. [\[2\]](#) [\[3\]](#)

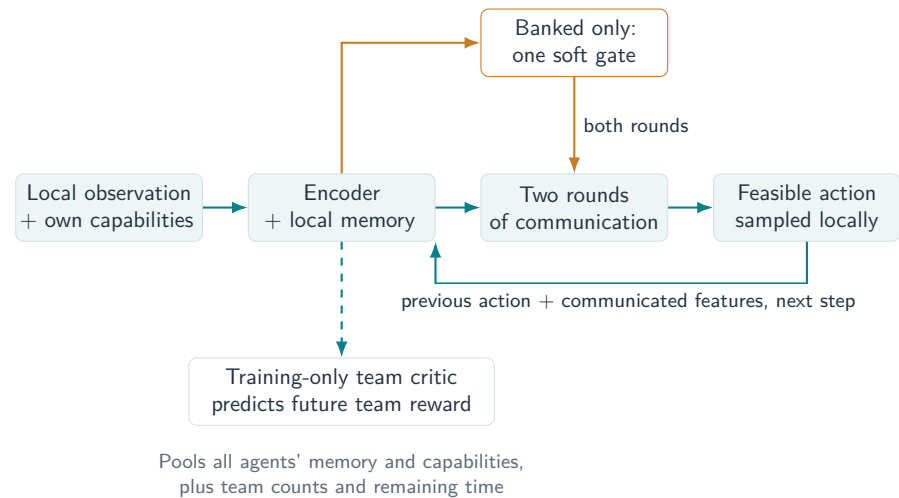
Attention and learning: GATv2 motivates listening that depends on the receiver. Generalized Advantage Estimation (GAE) helps convert experience into a learning signal. Deep Sets motivates combining agent features without depending on their order. These are building blocks, not evidence that our trained policies generalize. [\[4\]](#) [\[5\]](#) [\[6\]](#)

Evidence. E2: task recipes in `softrole/config.py`, original environments, and `softrole/RESEARCH.md`. Numbered links identify primary papers; E1–E5 are mapped to local records on page 10.

02 / ARCHITECTURE OVERVIEW

One policy, different capabilities

SoftRole gives every agent the same actor architecture. The **actor** chooses actions from local information. Each agent has its own memory, so shared weights do not mean identical behavior. The banked model adds a learned blend of communication transforms; the shared model supplies the baseline for judging whether that blend helps.



Read the diagram as a repeated cycle

The agent summarizes its current view and combines it with memory. It sends a short message and weighs what it hears, twice, then chooses a possible action. Memory, communication features and action feed the next step. Past observations can influence decisions without assuming perfect recall.

The **critic** predicts how much team reward remains. It provides a training signal, like an estimate against which the outcome is compared. Its output never enters the actor’s decision. Its loss can still improve the shared encoder and memory during training.

Three models, two different comparisons

	HetNet Real	SoftRole shared	SoftRole banked
Information	Typed observations	Own abilities; merged agent counts	Same as shared
Communication	Class-dependent real vectors	One transform; binary messages	Four blended experts; binary messages
Learning path	Released learner	Team critic and team credit	Same as shared
Observation copies	Legacy behavior	Independent copies	Independent copies

Shared versus banked is our closest test of conditional communication weights. Banked also has more parameters. HetNet differs in several pipeline components.

The network dimensions do not depend on team size. That makes new teams executable with the same weights; it does not guarantee successful transfer. No explicit P/A label or agent identity selects parameters in the new actor. Own capabilities still correlate with physical class in ordinary training.

Evidence. E2: SoftRoleNet, CommunicationRound, adapt_observation; original hetgat/uavnet.py. “Independent copies” prevent overlapping observation windows from accidentally modifying one another.

03 / COMPONENTS

What each part actually does

1. Observation — define what is knowable. Each agent receives the existing position encoding, three sensory channels (outside-map, target, total number of agents), and two capability flags $\kappa = (\text{sense}, \text{actuate})$. Agent *counts* are added across classes, so two agents in one cell remain two. Neighbor class labels are removed. The input has 702 numbers in PP/PCP and 254 in FC. Losing sensing masks the three sensory channels; location information, movement, memory and radio remain. Own sensor health is supplied, not discovered by the policy.

2. Encoder and LSTM — summarize and remember. A learned encoder reduces the input to 128 features. An LSTM, a neural network with memory, combines these with 16 previous communication features and six previous-action indicators. It maintains a 64-number hidden state and a 64-number internal memory state. The input width is $128 + 16 + 6 = 150$. This is a learned summary, not a guaranteed map of everything previously seen.

3. Soft gate and banks — blend communication weights. In the banked model, memory and capabilities produce four nonnegative weights summing to one. For example, $(0.55, 0.25, 0.15, 0.05)$ blends four expert transforms in those proportions:

$$B(x, g) = \sum_{k=1}^4 g_k (W_k x + b_k), \quad g_k \geq 0, \quad \sum_k g_k = 1.$$

Here x is a feature vector, W_k and b_k are an expert's learned matrix and bias, and g_k is its mixing weight. One deterministic gate is computed per agent-step and reused in both rounds. "Deterministic" means the same inputs give the same gate; the messages and actions can still be random. The shared baseline uses one actual transform. Expert indices are not established semantic roles such as scout or rescuer.

4. Binary messages and attention — speak briefly, listen selectively. Each round sends one 16-bit payload per agent. A learned score u gives a bit a probability $\sigma(u)$ of being 1; σ is the sigmoid function. The same sampled payload reaches all permitted receivers and all four attention heads. Heads are four learned ways of reading a message, not four extra transmissions. Encoding uses the sender's gate; decoding and attention use the receiver's gate. A zero-message "null" choice lets an agent allocate attention to no incoming information. With no neighbors the received aggregate is exactly zero. Round 1 produces 64 features; round 2 produces 16.

5. Action head — respect physical abilities. The final 16 features produce probabilities for four moves, staying still, and capture/extinguish. If actuation is unavailable, the special action has exactly zero probability. The two rounds use **32 logical payload bits per agent-step**; that excludes packet headers and delivery overhead.

6. Team critic and update — learn from the whole team's outcome. The critic averages agent features and adds team counts and remaining time, so permuting the roster does not change its prediction. Rewards are summed over physical agents. At each time step, all agents' action scores use the same team advantage: how much better or worse the outcome was than expected. Gradients, which indicate how to adjust learned weights, are summed across complete episodes, divided once by the global episode count, clipped once, then used for one RMSprop update.

Training settings: actor coefficient 50, value coefficient 1; discount $\gamma = 1$; GAE $\lambda = 0.95$; gradient paths cut every 5 steps, while memory is retained; learning rate 10^{-4} ; RMSprop $\alpha = 0.97$, $\epsilon = 10^{-6}$; clip norm 0.75. Binary backpropagation uses a biased straight-through sigmoid derivative. GAE with a learned critic and truncated recurrence are also approximations.

Evidence. E2: `softrole/model.py`, `learning.py`, `train.py`, `config.py`. Current implementation uses deterministic gates; Gaussian roles, KL regularization and arrivals/departures within an episode are deferred.

04 / READING THE EVIDENCE

What the comparison can tell us

The resynced snapshot contains **27 runs and 37,566 completed epochs**, adding 10,688 epochs to the earlier same-day logs. It covers three tasks, three models and three seeds per combination. A **seed** starts a separate randomized training run. The 18 SoftRole runs record 631,310,527 environment steps and zero exposed sensor failures. These are fixed-team runs; their curves do not test adaptation.

Return adds all agents' rewards across one episode; higher, including less negative, is better within a task. **Success** is the fraction of episodes meeting the task objective. **Episode length** counts steps until success or the time limit, including unsuccessful episodes. Read length with success: fewer steps alone is not sufficient evidence of improvement. An **epoch** groups ten optimizer updates here.

A contextual comparison at the same epoch window

The following values average the last 50 epochs ending at the latest epoch available for every run of a task. Epochs are weighted equally within each run; the three seeds then receive equal weight. These windows compare update counts, not equal samples or compute.

Task / epochs	Model	Success (%)	Return	Steps
PP / 343–392	HetNet Real	99.71	-2.115	18.25
PP / 343–392	shared	100.00	-0.374	4.88
PP / 343–392	banked	100.00	-0.376	4.90
PCP / 949–998	HetNet Real	99.59	-2.112	17.94
PCP / 949–998	shared	99.99	-0.548	6.92
PCP / 949–998	banked	99.99	-0.638	8.61
FC / 976–1025	HetNet Real	40.73	-223.759	218.96
FC / 976–1025	shared	64.37	-143.426	184.44
FC / 976–1025	banked	48.58	-191.338	211.75

Rounding can show 100.00% despite rare failures. Table values are changing-policy training statistics, not held-out evaluation scores. "Steps" here means episode length.

Why the following plots use two horizontal axes

HetNet's printed cumulative sample counters repeatedly add already-cumulative within-epoch totals. They therefore overcount. We use its valid completed-epoch axis. SoftRole's step and episode counts reconcile, so its plots use recorded environment steps. The two columns' horizontal positions are not equal training budgets.

For the closer **shared versus banked** comparison, pages 5–7 use the last 50 completed epochs at or before a common sample budget. Within each seed, an epoch with twice as many episodes receives twice the weight; the three seed estimates are then averaged equally. This pools episodes without letting a faster seed dominate. Endpoints fall less than 0.08% below the stated budgets.

Observed progress is real in these logs. It does not isolate the effect of banks against HetNet, establish final convergence, or demonstrate robustness.

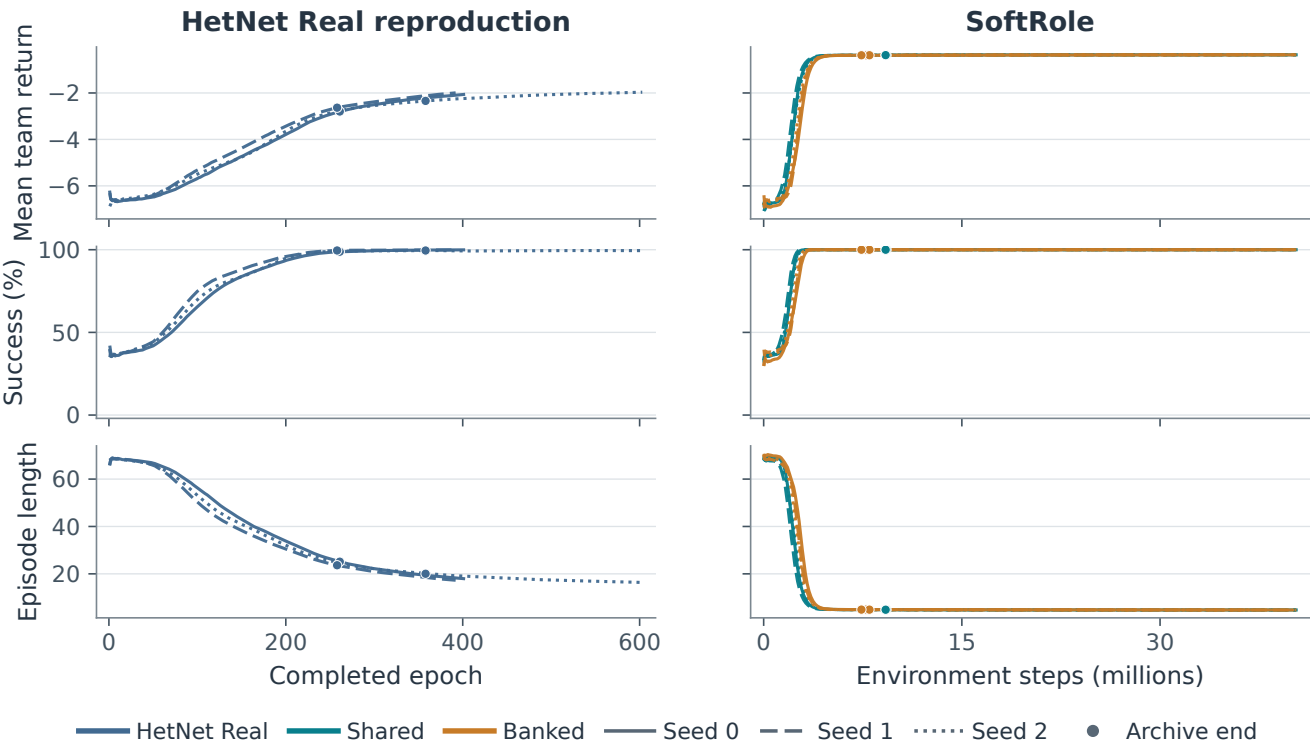
The pipelines differ in observations, binary versus real communication, recurrence and learning. Three seed traces show variability; they are not confidence intervals. New stdout extends the older structured archive, but the corresponding newer checkpoints and source archives have not been supplied.

Evidence. E1: current raw logs re-parsed; old byte prefixes and archived metric/configuration identities verified. Per-run windows and grouped statistics independently recomputed. Source lines and exact windows are retained in the audit.

05 / PREDATOR–PREY

PP: strong learning, little separation

Three sensing agents must all reach a stationary prey before 80 steps. This is the simplest nominal test: there is no separate actuation agent and no sensor loss.



Each line is one seed. Curves use trailing 50 epochs: equal-epoch means for HetNet; episode-weighted means for SoftRole. Dots mark the older structured-archive endpoint. Left and right use different training axes; see page 4.

SoftRole at 37 million environment steps

Variant	Success (%)	Team return	Episode steps
Shared	99.999	-0.356	4.74
Banked	99.995	-0.357	4.75

What has improved

Both SoftRole variants reach approximately 100% training success and about 4.75 steps per episode. At the same 37M-step budget their returns and lengths are close: banked is about 0.01 steps longer on average. Additional training has changed this comparison very little.

The HetNet reproduction also reaches high success: its own latest 50-epoch windows average **99.75%**, with return -2.009 and length **17.21**, improving from 18.85 steps in the previous snapshot. These latest windows end at different epochs across seeds; page 4 gives the common-epoch comparison.

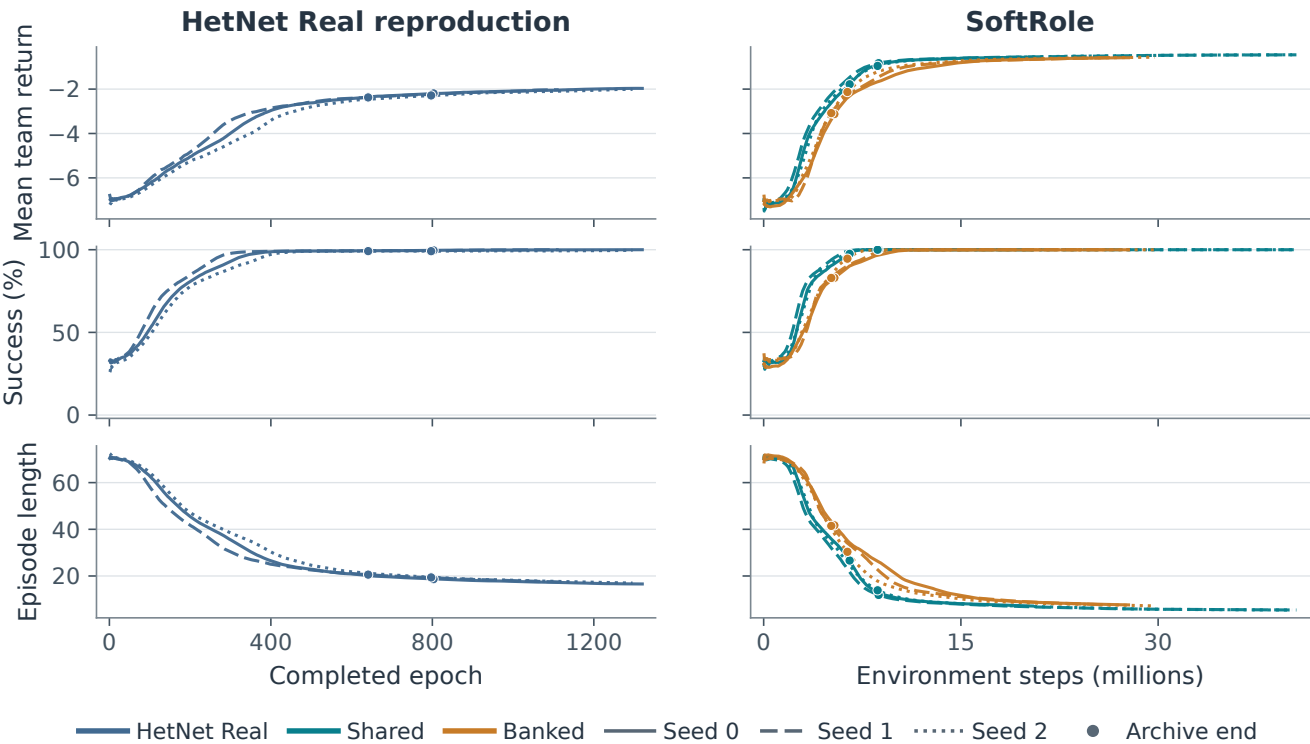
PP confirms that the implemented policies can learn the ordinary task. It supplies little evidence that four experts help beyond a shared transform.

Evidence. E1: summary.json, task=pp, common_sample and latest; all three seeds. Raw stdout paths and line numbers are in epoch_metrics.csv.

06 / PREDATOR–CAPTURE

PCP: success catches up; speed differs

Two sensing agents locate and reach the prey; one actuation agent must also reach and capture it. This is our primary domain for studying changed teams and sensor loss, although these training curves use the fixed, healthy team.



Each line is one seed; trailing 50-epoch means with the weighting from page 4. Dots mark the older archive endpoint. The runs end at different sample counts; the table compares a common budget.

SoftRole at 20 million environment steps

Variant	Success (%)	Team return	Episode steps
Shared	99.995	-0.552	7.00
Banked	99.989	-0.645	8.75

The positive result and the remaining gap

At the earlier 5M-step comparison, shared/banked success was **91.07% / 81.89%**. At 20M it is approximately 100% for both, using the same episode-weighted method. Banked has largely closed the earlier success gap.

Shared remains faster: **7.00 versus 8.75 steps**, or **20.0% fewer** using $1 - 7.0010/8.7523$. The gap has narrowed from 26.7% at 15M. Seed lengths span 6.85–7.20 for shared and 8.43–8.96 for banked. Shared also earns the less-negative return. This is an observed efficiency advantage, not a significance claim.

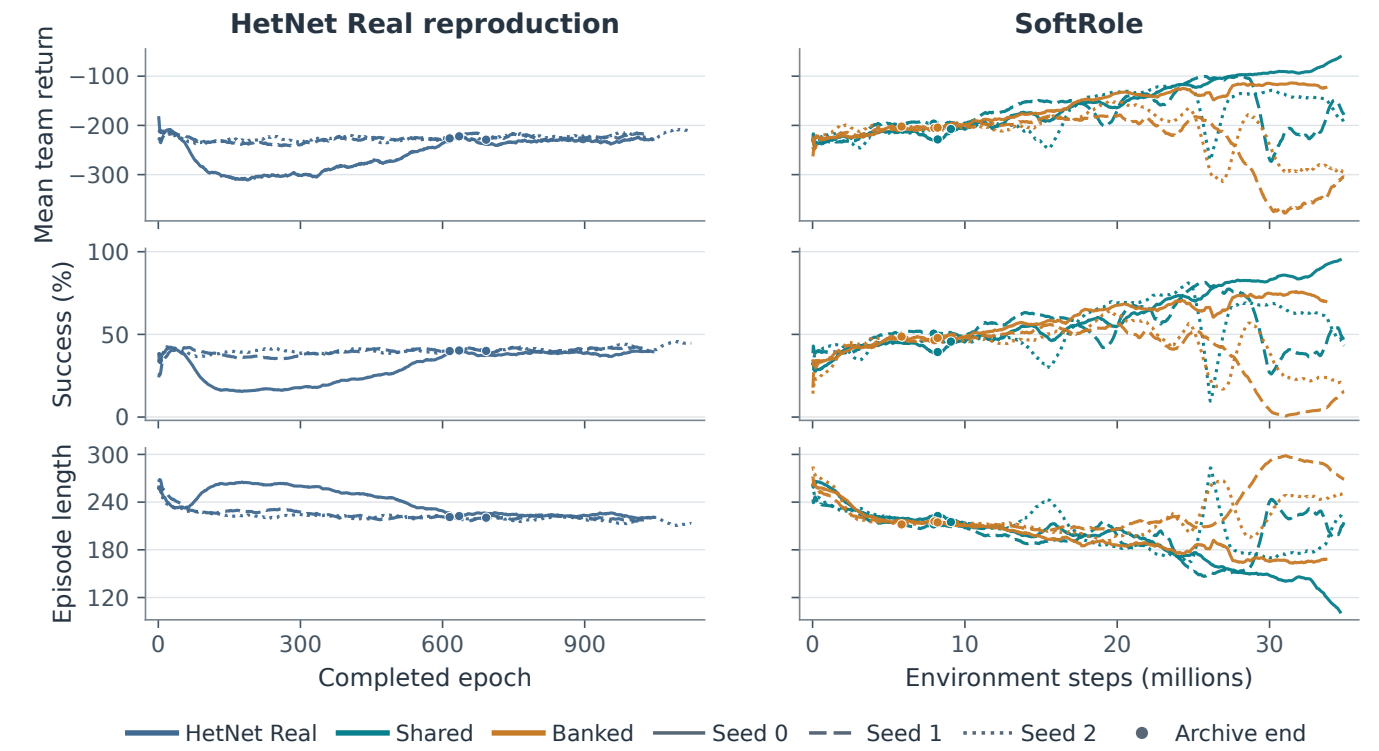
HetNet’s own latest windows average **99.88%** success, return **−2.003**, and length **16.99**. Its task learning is clear; pipeline differences prevent assigning the gap to any single SoftRole component.

Banked learns nominal PCP successfully and the speed gap is narrowing. Shared retains the observed efficiency advantage at 20M steps.

Evidence. E1: task=pcp, prior_common_sample, common_sample, per-run windows and latest. Reported speed reduction is a ratio of equal-seed mean episode lengths.

FC: progress with serious qualifications

The team must extinguish all fire within 300 steps. The new logs show wide seed variation; a confirmed simulator defect also limits interpretation.



Three seeds per variant; trailing 50-epoch means. Common-budget estimates below should not be replaced by the unequal-budget endpoints. Lower episode length includes failures at the 300-step limit.

SoftRole at 33.5 million environment steps

Variant	Success (%)	Team return	Episode steps
Shared	63.91	-143.273	178.35
Banked	32.97	-251.131	233.31

At this budget, shared success is **30.94 percentage points higher**. Seed ranges are very broad: **39.45–90.67%** for shared and **4.52–70.19%** for banked. HetNet’s latest windows average **41.98%** success, return **–219.862**, and length **217.21**.

The new tail exposes further instability. Latest shared seed 0/1/2 success is **95.15/46.22/43.12%**; banked is **69.92/15.53/19.90%**. Banked seeds 1 and 2 regress substantially. Five FC SoftRole runs reach epoch 1,400; banked seed 0 ends at 1,354. These latest windows have unequal sample counts.

Environment defect: skipped boundary fronts create spurious origin fires and rewards. The prior audit found this placeholder in 945/1,000 passive resets by step 300 and 138/144 older frozen-policy outcomes. This limits physical interpretation; it does not establish the cause of the late regressions.

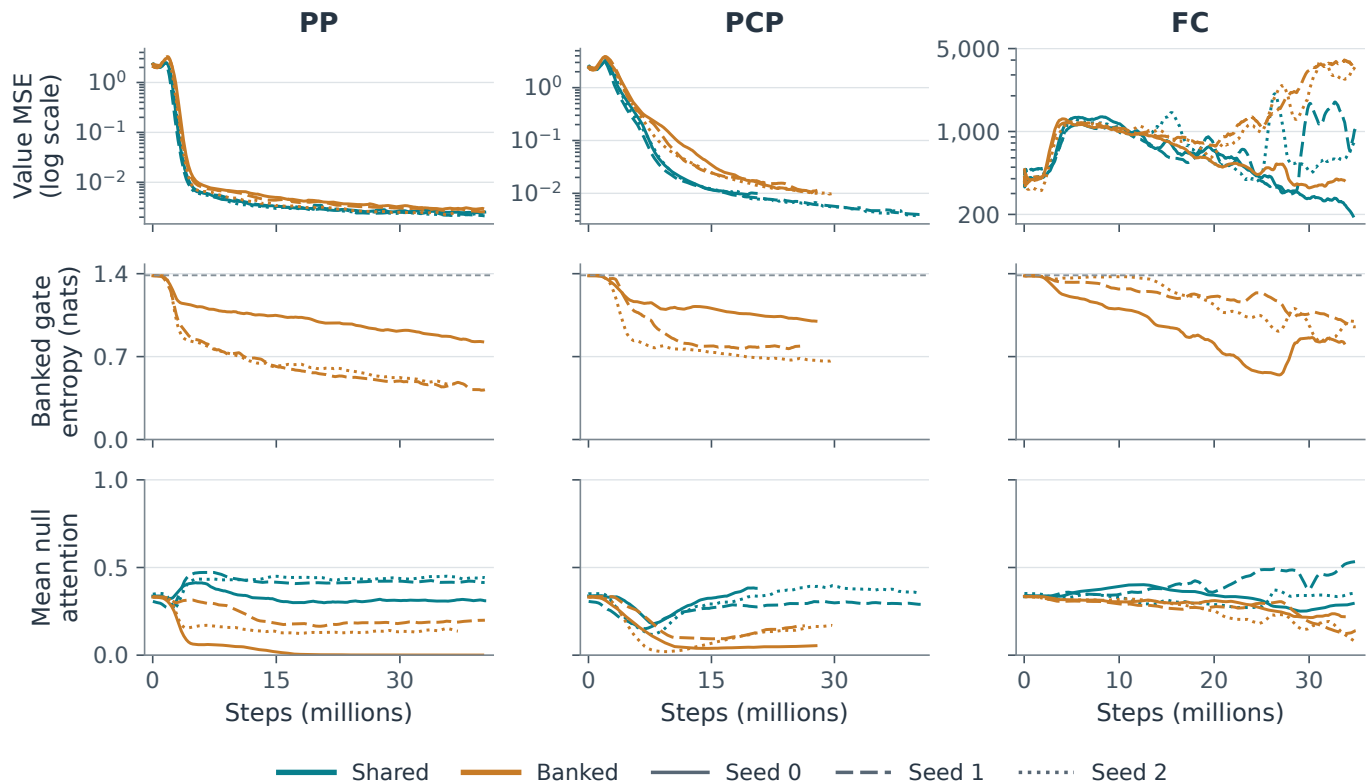
FC shows learning, variability and a simulator problem. It is not clean evidence of physical firefighting quality or a settled architecture ranking.

Evidence. E1: FC common-budget and latest per-seed windows. E3: simulator_probe.json and policy_panel/summary.json. The earlier simulator/policy audit was not rerun for this document.

08 / LEARNING DIAGNOSTICS

What is happening inside training?

Performance curves say whether behavior improves. The following diagnostics help locate changes in training, but they cannot by themselves identify why a policy succeeds or fails.



SoftRole only. Three seeds per variant; episode-weighted trailing 50-epoch means. Value error uses a logarithmic vertical scale. Gate entropy is shown only for banked; shared has one transform.

Value MSE: how far the critic is from its training target. MSE means mean squared error: prediction errors are squared before averaging. Lower is closer to the current target, which itself depends on a learned critic and GAE. Latest FC banked seed 1/2 errors are about 3,609/3,642, alongside 15.53/19.90% success. This association is a reason to inspect training, not proof that critic error caused the regressions.

Gate entropy: how spread out the four expert weights are. Zero would mean all weight on one expert; the maximum $\log 4 \approx 1.386$ means equal weights. A low value alone does not show useful specialization, and a high value does not show failure. The experts' learned transforms and actual policy behavior also matter.

Null attention: how much attention goes to receiving zero. Within each episode, it averages the no-message weight over steps, agents, rounds and heads; episodes then receive equal weight. An isolated receiver necessarily gives null all its weight. Nonzero attention to neighbors does not prove their messages cause better behavior; a matched communication-removal experiment is needed.

These diagnostics guide experiments; they do not prove roles, communication benefit or adaptation.

Loss scales differ across pipelines and cannot rank policies. The latest SoftRole epoch logs omit action entropy and gradient norms; older audits do not supply current values.

Evidence. E1: epoch fields value_loss_per_episode, gate_entropy, alpha_null, and latest FC per-seed windows. E2: their computation in rollout.py and learning.py.

09 / ADAPTATION AND STATUS

The main research question is still open

The large training snapshot has **fixed teams and no sensor-loss exposure**. It establishes ordinary task learning. A separate, small PCP pilot checks the failure protocol and illustrates why success after a scheduled failure is not automatically evidence of recovery.

What the sensor-loss pilot actually did

One nominally trained checkpoint per model, both seed 0, was chosen using the first archived checkpoint reaching 4M steps: shared had 4,265,196 steps, banked 4,279,220. Both were epoch 200. Twenty common scenarios scheduled one sensing-agent failure at a step from 10–30. Each had a matched **sham**: the same scenario, victim, event schedule and random streams, with sensor loss suppressed. Weights were frozen.

Model	Sham successes	Failure successes	Mean return sham / failure	Exposed to loss
Shared	15/20	14/20	-3.2300 / -3.4575	15/20
Banked	15/20	16/20	-3.6575 / -3.6350	18/20

Shared changes by −5 percentage points and banked by +5; each is **one scenario out of twenty**. That does not establish model superiority or a benefit from sensor loss. Some scenarios finish before the event: 5/20 shared and 2/20 banked. Thus the two policies experience loss on different subsets even though the assigned panel is the same.

Among exposed victims, **14/15 shared and 18/18 banked** had already directly seen the target; **8/15 and 11/18** had reached it. This pilot mostly removes sensing after discovery. Prior sight is not guaranteed retained knowledge, and an agent already at the target may still help through messages. Full-panel outcomes remain primary; exposed-only statistics need their denominators.

What is implemented, and what remains

Implemented: shared and banked actors; three released tasks; variable rosters between episodes; PCP sensor loss; frozen evaluation with separate random streams; matched shams; event exposure and censoring; gate-freeze and communication-removal controls. Earlier engineering validation recorded 178 passing tests. Those tests check implementation behavior, not learned robustness.

Next evidence needed: run the locked multi-seed PCP failure-training comparison, test mechanism controls, then evaluate frozen weights on held-out teams without tuning to those outcomes. Prespecify any harder/earlier supplemental event using training compositions. Resync FC checkpoints near the regression and evaluate fixed scenarios. A physical FC correction should be explicitly versioned and studied separately.

Deferred: Gaussian latent roles and KL penalties, arrivals/departures during an episode, physical FC sensor-loss semantics, and formal paired seed-level inference. The code's ability to execute a changed team is not a result about performance on that team.

Current evidence supports a working, auditable system and strong nominal PP/PCP learning. The added value of banks for capability changes remains an experimental question.

Evidence. E4: the 20-scenario pilot's manifests, paired reports and summary. E5: prior 178-test validation; no training, evaluation or runtime test suite was launched to prepare this PDF.

10 / TRACEABILITY

How to check every result

This document uses the latest local logs available on 30 September 2026. All inputs were read without modification. The accompanying folder contains editable LaTeX, figure code, exact comparison values, raw-log checks, evidence mappings and SHA256 hashes (fingerprints that detect changed files).

Snapshot coverage

Task	Model (three seeds each)	Last epoch range	SoftRole steps (M)
PP	HetNet Real	392–603	–
PP	SoftRole shared	2000–2000	40.29–40.30
PP	SoftRole banked	1837–2000	37.04–40.32
PCP	HetNet Real	1113–1320	–
PCP	SoftRole shared	998–1998	20.45–40.52
PCP	SoftRole banked	1287–1455	26.40–29.73
FC	HetNet Real	1025–1125	–
FC	SoftRole shared	1400–1400	34.64–34.89
FC	SoftRole banked	1354–1400	33.70–34.98

Ranges describe three endpoints, not uncertainty intervals. HetNet’s new cumulative stdout sample counts are not usable. Nine runs reach their configured targets: four SoftRole PP and five SoftRole FC. Targets are 2,000 epochs for PP/PCP and 1,400 for FC; reaching them is not proof of convergence.

Local evidence map

E1 — current training. logs_1/*.out, logs_sr/*.out; this document’s latest_snapshot/{summary.json,epoch_metrics.csv ,provenance.json}. Each summary run names its raw log; the CSV names its source line. review_results.json checks raw values and arithmetic. Earlier comparisons use analysis/training_2026-09-30_resync/summary.json.

E2 — architecture and metrics. softrole/{env,model,learning,rollout,train,config}.py, softrole/RESEARCH.md, original model and simulator sources. evidence_index.json expands paths and lookup instructions.

E3 — FC audit. analysis/fc_audit_2026-09-30/: simulator_probe.json, log_findings.json, policy_panel/summary.json. These preserve the earlier diagnostic panels; they are separate from the newer training tail.

E4 — sensor pilot. runs/sensor_failure_validation_20260930/pilot/: pilot.json, scenarios.json, summary.json and four condition reports. **E5 — engineering record.** Repository AGENTS.md; dated checks are not represented as newly executed experiments.

Primary references

[1] Seraj et al. (2022), *Learning Efficient Diverse Communication for Cooperative Heterogeneous Teaming*. AAMAS.

[2] Howell et al. (2023), *Generalization of Heterogeneous Multi-Robot Policies via Awareness and Communication of Capabilities*. CoRL.

[3] Fu et al. (2025), *Capability-Aware Shared Hypernetworks for Flexible Heterogeneous Multi-Robot Coordination*. CASH.

[4] Brody et al. (2022), *How Attentive are Graph Attention Networks?* GATv2.

[5] Schulman et al. (2016), *High-Dimensional Continuous Control Using Generalized Advantage Estimation*.

[6] Zaheer et al. (2017), *Deep Sets*. Permutation-invariant aggregation.

[7] Agarwal et al. (2021), *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. Evaluation uncertainty; motivates caution with a few runs.

Evidence. All paper links are clickable. input_manifest.json records local input identities; comparison_values.csv gives unrounded numbers; validation.json records this document’s build and checks. No new performance experiments are presented.